

Student Name:	Devika
Date:	27-Aug-2026
Duration:	90 Minutes
Maximum Marks:	100
Level:	Foundation → AI Testing Readiness

Instructions

- Answer all questions. Show Python code wherever code is requested.
- Use Python 3 syntax. Do not use AI tools, ChatGPT, Copilot, or internet search during the test.
- For output-prediction questions, write the exact output and briefly explain why.
- Focus on correctness, data types, mutability, assignment behavior, and clean reasoning.

Section A — Rapid-Fire Fundamentals (20 × 2 = 40 marks)

1. Define a variable in Python. Why is Python called dynamically typed?

Answer: A variable is a name that refers to a value stored in memory.

Ex: name = "Python"

Python is called dynamically typed because you don't have to specify explicitly the datatype.
Python determines the type while the program is running.

2. Differentiate between =, ==, and is.

Answer: = assigns a value to a variable.

== checks whether two values are equal.

is checks whether two variables refer to the same object in memory.

Example: x = 10

x == 10 # True

x is 10 # checks object identity

3. List Python's built-in primitive data types covered in class and give one example of each.

Answer: int → 10

float → 3.14

complex → 2 + 3j

bool → True

str → "Hello"

4. List at least five non-primitive/collection data types in Python.

Answer: List → [1, 2, 3]

Tuple → (1, 2, 3)

Set → {1, 2, 3}

Dictionary → {"name": "John"}

Range → range(5)

5. What is multiple assignment? Give one example.

Answer: Multiple assignment means assigning values to multiple variables in one statement.

Ex: a, b, c = 10, 20, 30

6. What is tuple unpacking? Give one example.

Answer: Tuple unpacking means assigning the elements of a tuple to separate variables.

Ex: numbers = (10, 20, 30)

a, b, c = numbers

7. Explain mutable vs immutable objects with two examples of each.

Answer: Mutable objects can be changed after they are created.

Examples: list, dictionary.

Immutable objects cannot be changed after they are created.

Examples: string, tuple.

8. What happens when you execute `x = 10` followed by `x = 'AI'`?

Answer: First, x refers to the integer 10. Then x is reassigned to the string "AI".

Python allows this because it is dynamically typed.

9. How do you check the data type of a variable?

Answer: Use the `type()` function.

ex: x = 10

`print(type(x))`

10. How do you convert the string `'100'` into an integer?

Answer: Use the `int()` function.

Ex: `x=int('100')`

11. Give three useful string methods and explain what each does.

Answer: `upper()` → Converts all letters to uppercase.

`lower()` → Converts all letters to lowercase.

`strip()` → Removes whitespace from the beginning and end.

12. Give three list methods and explain what each does.

Answer: `append()` → Adds an item to the end of a list.

`Remove()` → Removes a specified item from a list.

`pop()` → Removes and returns an item from a list.

13. Give three dictionary methods and explain what each does.

Answer: `keys()` → Returns all the keys.

`values()` → Returns all the values.

`items()` → Returns all key-value pairs.

14. Give three set methods and explain what each does.

Answer: `add()` → Adds an item to a set.

`remove()` → Removes an item from a set.

`union()` → Combines the elements of two sets.

15. Give two tuple methods and explain what each does.

Answer: `count()` → Counts how many times a value appears in a tuple.

`Index()` → Returns the position of the first occurrence of a value.

16. What is indexing? Give one example using a string.

Answer: Indexing means accessing an individual element using its position.

Python starts indexing from 0.

Ex: text = "Python"

```
print(text[0])
```

17. What is slicing? Give one example using a list.

Answer: Slicing means getting a portion of a sequence using a start and end index.

Ex: numbers = [10, 20, 30, 40, 50]

```
print(numbers[1:4]). o/p:20,30,40
```

18. Why are strings immutable in Python? What does that mean practically?

Answer: Strings are immutable because their contents cannot be changed after they are created. Practically, you cannot change an individual character directly.

Ex: text = "Python"

```
text[0] = "J" # Error
```

19. What is the difference between a list and a tuple?

Answer: A list is mutable, so its elements can be changed. A tuple is immutable, so its elements cannot be changed.

```
list = [1, 2, 3]
```

```
tuple = (1, 2, 3)
```

20. What is the difference between a dictionary and a set?

Answer: A dictionary stores data as key-value pairs,

while a set stores unique values without key-value pairs.

Ex: dictionary = {"name": "John", "age": 25}

```
set = {10, 20, 30}
```

Section B — Predict the Output (10 × 3 = 30 marks)

1. What is printed? Explain whether y changes.

```
x = 10
y = x
x = 20
print(x, y)
```

Output / Explanation: 20 10

Explanation: y gets the value 10. Changing x to 20 does not change y.

2. What is printed?

```
a = "Python"
b = a
a = "AI Testing"
print(a)
print(b)
```

Output / Explanation: AI Testing

Python

Explanation: b still refers to "Python". Reassigning a does not change b.

3. What is printed?

```
x, y, z = 10, 20, 30
print(x + y + z)
```

Output / Explanation: 60

Explanation: 10 + 20 + 30 = 60.

4. What is printed?

```
items = [10, 20, 30]
items.append(40)
print(items)
print(len(items))
```

Output / Explanation: [10, 20, 30, 40]

4

Explanation: append(40) adds 40 to the end. The list now has 4 items.

5. What is the resulting dictionary?

```
d = {"name": "Vishwa", "role": "Tester"}
d["experience"] = 10
print(d)
```

Output / Explanation: {'name': 'Vishwa', 'role': 'Tester', 'experience': 10}

Explanation: A new key-value pair, "experience": 10, is added to the dictionary.

6. What is printed?

```
s = {1, 2, 2, 3, 3, 3}
print(s)
print(len(s))
```

Output / Explanation: {1, 2, 3}

3

Explanation: A set stores only unique values, so duplicate 2s and 3s are removed.

7. What is printed?

```
text = "AI Testing"
print(text.lower())
print(text.upper())
print(text.replace("Testing", "Engineering"))
```

Output / Explanation: ai testing

AI TESTING

AI Engineering

Explanation: lower() → lowercase

upper() → uppercase

replace() → replaces "Testing" with "Engineering"

8. What is printed?

```
nums = [1, 2, 3, 4, 5]
print(nums[0])
print(nums[-1])
print(nums[1:4])
```

Output / Explanation: 1

5

[2, 3, 4]

Explanation: nums[0] → first item → 1

nums[-1] → last item → 5

nums[1:4] → items from index 1 up to, but not including, index 4.

9. What is printed? Explain the reason.

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
print(b)
```

Output / Explanation: [1, 2, 3, 4]

[1, 2, 3, 4]

Explanation: a and b refer to the same list object. Therefore, when b.append(4) changes the list, the change is visible through a too.

10. What is printed?

```
x = 10
x += 5
x *= 2
print(x)
```

Output / Explanation: 30

Explanation: x = 10

x += 5 → 15

x *= 2 → 30

Section C — Coding Challenges (5 × 4 = 20 marks)

1. Create variables for: student name, age, course name, test score, and whether the student passed. Print each variable together with its data type.

```
Code: student_name = "Devika"
      age = 100
      course_name = "Python for AI Testing"
      test_score = 85
      passed = True
      print(student_name, type(student_name))
      print(age, type(age))
      print(course_name, type(course_name))
      print(test_score, type(test_score))
      print(passed, type(passed))
```

2. Write a program that stores 10 Python/AI-testing tool names in a list. Add one new tool, remove one existing tool, sort the list, and print the final list.

Code:

```
tools = ["Python", "PyTest", "Selenium", "Robot Framework", "Postman", "JMeter", "Playwright", "Cypress", "TensorFlow", "PyTorch"]

tools.append("OpenAI")
tools.remove("JMeter")
tools.sort()

print(tools)
```

Output: ['Cypress', 'OpenAI', 'Playwright', 'Postman', 'PyTest', 'Python', 'PyTorch', 'Robot Framework', 'Selenium', 'TensorFlow']

- `append("OpenAI")` adds a new tool to the end of the list.
- `remove("JMeter")` removes JMeter from the list.
- `sort()` arranges all tools in alphabetical order.
- `print(tools)` displays the final list.

3. Create a dictionary representing a test case with keys: test_id, test_name, status, priority, and execution_time. Update status from 'Not Executed' to 'Passed'.

Code:

```
test_case = {"test_id": "TC001", "test_name": "Login Test", "status": "Not Executed", "priority": "High", "execution_time": 5}

test_case["status"] = "Passed"

print(test_case)
```

Output:

```
{'test_id': 'TC001', 'test_name': 'Login Test', 'status': 'Passed', 'priority': 'High', 'execution_time': 5}
```

4. Given a string containing a student's name and course, use string methods to remove extra spaces, convert the text to a consistent case, and print the cleaned result.

Code:

```
text = " Devika Python for AI Testing "  
  
cleaned_result = text.strip().lower()  
  
print(cleaned_result)
```

Output: devika python for ai testing

Explanation: `strip()` removes extra spaces from the **beginning and end** of the string.

`lower()` converts all letters to **lowercase**.

5. Create a tuple containing five AI-testing concepts. Demonstrate indexing and slicing, and explain why you cannot directly change one tuple element.

Code:

```
concepts = ("AI", "Testing", "Python", "Automation", "Data")  
  
print(concepts[0])  
  
print(concepts[1:4])
```

Output: AI

('Testing', 'Python', 'Automation')

Explanation:

- **Indexing:** `concepts[0]` gives the first item.
- **Slicing:** `concepts[1:4]` gives items from index 1 to 3.
- **Tuple:** A tuple cannot be changed because it is **immutable**.

Example:

`concepts[0] = "Machine Learning"`. O/P: Will get error because immutable

Section D — AI Testing Bridge Challenge (10 marks)

An AI testing framework receives test data from different sources. Some values arrive as strings, some as integers, some as lists, and some as dictionaries. Design a small Python program that accepts a variable **test_input**, prints its value and data type, and behaves differently for at least three different data types. Explain why understanding Python data types is important when building AI/LLM testing automation.

Expected elements: type checking, clean variables, at least 3 branches, meaningful output, and readable code.

Bonus — Interview Question (Optional, +5)

You are testing an AI application and the response payload contains a Python dictionary with nested lists and dictionaries. Explain why choosing the correct Python data type and understanding mutability can matter when modifying test data, expected results, and test fixtures.

Instructor Evaluation Focus

Conceptual clarity 25% • Output reasoning 25% • Coding correctness 35% • AI-testing connection 15%

